Divide and Conquer in Decentralized AI Training Coordination

Two students • 24-week plan • Source analysis, controlled experiments and one mitigation

1. Background

Decentralized AI platforms allow independent participants to pool computational resources and train models together. Hivemind uses a distributed hash table (DHT) for peer discovery and organizes parameter averaging in temporary groups [1]. Discovery, group formation, admission, timeouts and recovery determine which peers participate in each aggregation.

Byzantine-robust aggregation rules are designed to limit the influence of malicious model updates. Krum requires $n > 2f + 2$, where n is the number of contributing workers and f an upper bound on malicious workers. With seven workers, this count condition permits at most two malicious workers. Krum also requires statistical assumptions about honest gradients, including their variability [2]. In training with temporary groups, these assumptions concern the contributors in each group. Discovery and admission determine who can join, while timeouts and dropouts affect who actually contributes.

2. Literature gaps

Prior work studies malicious model updates (Krum [2]) and attacks on peer selection (GRANITE [3]). The remaining questions for the platforms in this study concern how discovery, admission, timeouts and recovery work together:

Effects of coordination attacks. Which forged or delayed messages change who contributes? It remains to be established whether these changes concentrate malicious peers, exclude honest peers or interrupt aggregation, and how they affect training.

Use of P2P protections. IPFS and Ethereum use protections for peer discovery and communication [4–7]. Their use in training coordination needs evaluation: which attacks do they prevent, and how much delay, traffic or rejection of honest peers do they cause? WP1 will check these gaps against related work on the selected platforms.

3. Goals and research questions

Goal: Test whether manipulating coordination messages can change group participation or disrupt training, and measure the effect and cost of one mitigation.

RQ1. Manipulation: Which forged, misleading or delayed messages change discovery, timing, membership, roles or weights, and which checks prevent this?

RQ2. Influence and cost: How does the attacker fraction or weight in each group change with the number of malicious peers, extra identities, group size and messages manipulated?

RQ3. Effects: Do attacks exclude honest peers, stop aggregation rounds or reduce model accuracy? Can individual peers be affected while overall training throughput changes little?

RQ4. Mitigation: Which IPFS/libp2p or go-ethereum protections apply, and how does one implemented mitigation change attack effectiveness, latency and resource use?

4. Scope and experimental approach

Platforms

Compare three open-source AI platforms that use DHT and libp2p technologies. Hivemind is the starting example. Select the primary platform and two comparison platforms with the supervisor in WP1. Each must have inspectable code and a coordination protocol relevant to collaborative training. Implement attacks and one mitigation in the primary platform. Review the other two platforms' code and test selected mechanisms where practical.

Assess protections from IPFS/libp2p and go-ethereum: binding routing identifiers to authenticated peers, checking message signatures and freshness, and diversifying peer selection [4–7]. Check what each mechanism protects. Authentication verifies who sent a message. It does not prove that its contents are true or stop one attacker from creating several identities.

Threat model

Specify how many peers and identities the attacker controls. These peers may send false coordination data, replay accepted records, withhold replies or delay their own messages. Honest peers, their private keys and the testbed hosts remain uncompromised. If experiments also delay traffic between honest peers, report this as a separate network-attacker capability.

Controlled evaluation

Run the platforms on a private testbed. Log both admitted peers and peers whose updates are actually aggregated. Compare each attack with the same peers behaving honestly, and repeat with the mitigation enabled. Include ordinary delays and peer departures as controls. Use the same workloads and several random seeds. Vary one factor at a time before testing combinations. Report variation across runs.

Measure	Definition / comparison
Participation	Global fraction $\alpha = k/N$: k malicious peers among N total. Group fraction $\beta = a/n$: a malicious contributors among n total contributors. Also measure attacker weight when aggregation uses weights.
Attack effect and cost	Change in β compared with honest runs and number of rounds affected. Attack cost in identities, messages, bytes, computation and time.
Availability	Completed/failed rounds, honest-peer inclusion, matchmaking and aggregation latency, retries, recovery time and throughput.
Learning	Loss and task accuracy over optimizer steps and wall-clock time, using a small reproducible workload. Tie changes to logged aggregation events.
Mitigation overhead	CPU, memory, network traffic and latency. Effects on honest churn, admission and successful aggregation.

Use the measured honest group distribution as the baseline: groups need not be random samples of the network. Measure training effects directly. Exceeding an aggregation rule's worker bound alone does not demonstrate harm. Krum illustrates such a bound and is not Hivemind's built-in aggregator.

5. Work packages and milestones

The work packages overlap. Literature review and thesis writing continue throughout the 24 weeks.

WP1: Foundation and platform selection | Weeks 1–3

Review robust aggregation, peer-sampling attacks and P2P discovery security. Select three platforms, record their versions and define the attacker model. Run the primary platform and log its aggregation decisions.

Expected output: A literature review, platform selection and working baseline.

Fallback: Replace a primary platform that cannot run locally. For an unavailable comparison platform, document its code and deployment limits.

WP2: Testbed and protocol analysis | Weeks 3–7

Make peer count, group size and peer departures configurable. Trace discovery, admission, roles, weights, timeouts and recovery. Record senders, proposed group members and actual contributors. Measure group composition with all peers behaving honestly.

Expected output: Repeatable baseline runs and a comparison of coordination messages and checks across the three platforms.

Fallback: Use fewer peers and small CPU workloads if hardware is limited.

WP3: Attack implementation and controls | Weeks 6–11

Test false discovery records, misleading availability claims, replay or selective delay, as supported by the code review. First keep model updates honest to isolate coordination effects. Then, where needed, hold a model-poisoning attack fixed and compare its effect with and without coordination manipulation.

Expected output: Attack implementations and logs showing which manipulations work, which fail and why.

Fallback: If group concentration fails, test whether the same messages cause exclusion, delays or failed rounds.

Midterm milestone | Weeks 10–12

Present the threat model, protocol comparison, baseline and one tested attack. Choose the mitigation and remaining experiments with the supervisor.

Two-student collaboration

Student A can lead the testbed and attacks. Student B can lead the platform comparison and P2P defence review. Both develop the baseline, implement and evaluate the mitigation, analyse results and write the thesis.

5. Work packages and milestones (continued)

WP4: Attack effect and cost | Weeks 11–16

Vary attacker fraction, group size and message manipulation. Measure attacker concentration, honest-peer exclusion and failed rounds. Compare attacks on one peer or group with attacks on all peers. Run a small training task to measure loss, accuracy and throughput.

Expected output: Measurements showing how attack effects change with attacker resources and workload.

Fallback: Restrict the sweep to one attack. If training measurements are inconclusive, report the participation and availability results with their limits.

WP5: Mitigation design, implementation and evaluation | Weeks 15–20

Choose a mitigation after reviewing IPFS/libp2p and go-ethereum protections. Implement it in the primary platform. Repeat attack and honest runs with it enabled and disabled, including honest joins, departures and delays.

Expected output: One mitigation, measurements of its effect and overhead, and any attacks it still allows.

Fallback: Implement a focused message-validation or identity-binding check if a larger protocol change is too costly.

WP6: Reproducible artifact and thesis | Weeks 19–24

Package the testbed, attacks, mitigation, analysis scripts and raw traces. Record dependencies, seeds, configurations and experiment identifiers. Complete the thesis and final presentation.

Expected output: Code and instructions to repeat the experiments, the thesis and presentation.

Fallback: Drop optional experiments. Finish documenting and analysing the completed runs.

Completion milestones

M1 (by Week 12): Protocol comparison, honest baseline and tested attack.

M2 (by Week 16): Attack-effect and cost measurements, including attacks on individual peers.

M3 (by Week 20): One mitigation implemented and evaluated.

M4 (by Week 24): Experiment code, results, thesis and presentation.

Scope and research conduct

Use private testbeds and small training workloads. Label emulated delays separately from measured network delays. Public-swarm experiments require prior operator agreement. Discuss vulnerability disclosure with the supervisor.

6. Expected research outcomes

The thesis should establish:

- Which tested messages can change group membership, contribution weights or aggregation completion, and which checks block them.
- How attacker numbers and resources affect the attacker fraction within groups, honest-peer participation and training performance.
- How the mitigation changes attack success, latency, CPU, memory and network traffic.

Research deliverables

A comparison of three platforms, testbed, attack and analysis code, raw experiment traces, one mitigation implementation, and the thesis and presentation.

7. Learning outcomes

Students will learn to analyse DHT/libp2p protocols, define attacker capabilities, run controlled experiments, implement and evaluate a mitigation, and explain what their measurements do and do not show.

8. Desired qualifications

Two students with strong programming skills, preferably Python, and basic knowledge of computer networks and distributed systems. Familiarity with Linux and containers is beneficial. Network-security or distributed-systems coursework is an advantage. Basic machine-learning knowledge is helpful.

9. Selected references

[1] Hivemind contributors. Hivemind: Decentralized Deep Learning in PyTorch. Software and documentation.
Project repository

[2] P. Blanchard, E. M. El Mhamdi, R. Guerraoui and J. Stainer. Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent. NeurIPS, 2017.
Paper

[3] Y. Belal, M. Maouche and S. Ben Mokhtar. GRANITE: a Byzantine-Resilient Dynamic Gossip Learning Framework. arXiv:2504.17471, 2025, revised 2026.
Preprint, version 2

[4] S. Sridhar et al. Content Censorship in the InterPlanetary File System. NDSS, 2024. DOI: 10.14722/ndss.2024.23153.
Paper and publication page

[5] libp2p contributors. Kademlia DHT specification.
Protocol specification

[6] Ethereum devp2p contributors. Ethereum Node Records and Node Discovery Protocol v5. Specifications for signed records and authenticated discovery.
ENR specification · Discovery v5

[7] go-ethereum contributors. Peer discovery implementation, p2p/discover.
Implementation

Software references consulted 7 October 2026. Experiment reports will record the versions or commits used.